

HelixTranslate

HelixTranslate

Verifizierte Buchübersetzung - ehrlich konzipiert, niemals stille Ausweichlösung.

Zusammenfassung

HelixTranslate ist eine hochleistungsfähige, auf Go basierende E-Book-Übersetzungsplattform, die Bücher zwischen über 100 Sprachen mithilfe verifizierter LLM-Anbieter übersetzt. Sie verfügt über Echtzeit-WebSocket-Überwachung und eine strikte „No-Silent-Fallback“-Routing-Richtlinie, die laut scheitert, statt stillschweigend an Qualität einzubüßen.

Kurzbeschreibung

Universelles E-Book-Übersetzungstoolkit auf Go-Basis. Übersetzt FB2, EPUB, TXT, HTML, PDF und DOCX in über 100 Sprachen mit den leistungsstärksten verifizierten LLM-Modellen (über die LLMsVerifier-Brücke), unterstützt REST/HTTP-3- und gRPC-APIs, verteilte Verarbeitung sowie ein Echtzeit-WebSocket-Überwachungsdashboard.

Ausführliche Beschreibung

HelixTranslate ist ein unternehmensreifes, auf Go basierendes System zur Übersetzung ganzer Bücher zwischen Sprachen – nicht nur Absätze oder Ausschnitte, sondern vollständige Werke von Anfang bis Ende. Es parst und generiert mehrere E-Book-Formate (FB2, EPUB, TXT, HTML, PDF, DOCX), unterstützt über 100 Sprachen mit automatischer Spracherkennung und bietet sowohl CLI-Tools als auch API-Server (REST über HTTP/3, gRPC sowie einen WebSocket-Ereignisstream), sodass es sich gleichermaßen in einen Terminal-Workflow oder ein Service-Mesh integrieren lässt. Sein prägendes Merkmal ist die *Art der Modellauswahl*: Statt einen Anbieter fest zu verankern und darauf

zu hoffen, dass er stabil bleibt, überträgt HelixTranslate die gesamte Modellautorität an die LLMsVerifier-Brücke (pkg/bridge), die das stärkste *verifizierte* API-Modell auswählt und eine deterministische, nach Bewertung sortierte Fallback-Kette zurückgibt. Die Eignung eines Modells wird anhand eines gewichteten Scores aus Reaktionszeit, Codequalität, Funktionsumfang und Zuverlässigkeit bestimmt – das Modell, das Ihre Übersetzung übernimmt, hat sich seinen Platz also durch nachgewiesene Leistung verdient, nicht durch eine bloße Erwähnung in einer Konfigurationsdatei.

Entscheidend ist, dass das System eine „No-Silent-Fallback“-Regel direkt im Code verankert: Fehlt ein API-Anbieterschlüssel oder fordert ein Nutzer explizit einen nicht verfügbaren Anbieter an, gibt die Pipeline einen klaren Fehler zurück, statt heimlich auf andere Anbieter auszuweichen oder auf eine lokale Laufzeitumgebung zurückzugreifen und so zu tun, als sei alles in Ordnung – eine Vorgabe, die durch ein dediziertes Pre-Build-Gate und einen dazugehörigen Mutationstest abgesichert ist. Lokale Laufzeitumgebungen (Ollama, llama.cpp) wurden bewusst aus dem Standardpfad entfernt, damit ein schwächeres System niemals unbemerkt die Arbeit eines verifizierten Modells übernehmen kann. Um den Übersetzungskern herum ist ein Echtzeit-WebSocket-Überwachungssystem aufgebaut: Das Übersetzungs-CLI sendet typisierte Ereignisse an einen Monitoring-Server, der ein Live-Web-Dashboard antreibt, während entfernte SSH-Worker die Arbeitslast für verteilte Übersetzungen aufteilen. Darüber hinaus sorgen Mehrfachdurchläufe für Konsistenz, eine Qualitätsanalyse in der Vorbereitungsphase, Übersetzungscaching zur Kostenkontrolle bei langen Texten sowie eine bildgestützte Qualitätssicherung für eine durchgängig hohe Ergebnisqualität. Die gesamte Plattform folgt einem Anti-Bluff-Engineering-Prinzip: Tests müssen reale, für den Nutzer sichtbare Ergebnisse nachweisen, gestützt auf obligatorische Mutationstests statt auf grüne Häkchen, die nichts beweisen.

Warum wir es entwickelt haben

Um langformatige Bücher zuverlässig und *ehrlich* zu übersetzen – niemals eine „abgewertete, aber vorhandene“ Übersetzung auszuliefern. Die Grundannahme des Designs lautet: Eine fehlende oder nicht verifizierbare Übersetzung muss ein lauter, unmissverständlicher Fehler sein, und die Modellauswahl muss stets auf einen tatsächlich verifizierten Anbieter zurückgreifen, statt auf eine fest einprogrammierte Vermutung oder einen stillen lokalen Fallback.

Warum es ein Game-Changer ist

Die meisten LLM-Übersetzungspipelines versagen lautlos – sie greifen leise auf ein schwächeres Modell zurück, wechseln zu einem lokalen Runtime-System oder liefern nur Teilausgaben, während die Testsuite weiterhin grün leuchtet und niemand den Qualitätsabsturz bemerkt. HelixTranslate macht diesen gesamten Fehlermodus strukturell unmöglich: Die Modellauswahl ist verifizierungsgesteuert, die Fallback-Kette ist deterministisch und vollständig transparent, und „kein Schlüssel / kein verifiziertes Modell“ führt zu einem ehrlichen, harten Fehler statt zu einem stillen Achselzucken. Diese eine Designentscheidung verwandelt die Frage *„Wurde diese Übersetzung tatsächlich von einem leistungsfähigen, verifizierten Modell durchgeführt?“* von einer unüberprüfaren Hoffnung in eine Garantie, die das System in Ihrem Namen durchsetzt.

Was innovativ ist

- **Verifizierungsgesteuerte Modellweiterleitung** über die LLMsVerifier-Brücke – das stärkste *verifizierte* Modell wird automatisch ausgewählt, sodass Betreiber Absichten formulieren, nicht Anbieternamen, und niemals manuell einen Provider wählen, der möglicherweise nicht verfügbar ist.
- **Garantie gegen stille Fallbacks, in Code erzwungen** – vier explizite Weiterleitungszweige (Mock / expliziter Verifizierer / expliziter Anbieter / Brücken-Standard), die jeweils mit einem harten Fehler abbrechen, statt leise umzuschalten, sowie die gezielte Entfernung lokaler Runtimes aus dem Standardpfad, sodass es nichts Schwächeres gibt, *worauf* zurückgegriffen werden könnte.
- **Mechanische Durchsetzung** – ein CM-NO-LOCAL-RUNTIME-Pre-Build-Gate sowie ein dazugehöriger Mutationstest prüfen bereits zur Build-Zeit, dass auf dem Standardpfad niemals ein lokaler Runtime-Client erstellt wird: Die Garantie kann nicht verfallen, weil der Build fehlschlägt, wenn dies doch geschieht.
- **Deterministische, nach Bewertung geordnete Fallback-Kette** – Failover zwischen *verifizierten* Modellen ist erlaubt und vollständig transparent, ein prinzipieller Unterschied zum verbotenen stillen Fallback: Sie wissen stets, welches leistungsfähige Modell die Arbeit übernommen hat.
- **Echtzeit-Websocket-Überwachung** – getypte Übersetzungsereignisse werden live an ein Dashboard gestreamt, mit verteilten SSH-Workern, sodass ein Buchlängen-Job sichtbar und parallel abläuft, statt eine Blackbox zu sein.
- **Anti-Bluff-Testregime** – Mutationstests, Negativ-Assertions, Echt-System-Läufe und visuelle QS sorgen gemeinsam dafür,

dass „Tests bestehen“ niemals stillschweigend „Funktion tut nicht wirklich“ verschleiern kann.

Größte technische Herausforderungen & wie wir sie gelöst haben

- **Ehrliche Übersetzungspipeline garantieren (keine stille Verschlechterung).** Gelöst durch die Zentralisierung aller Modellautorität in der LLMsVerifier-Brücke, sodass es einen einzigen Entscheidungspunkt zur Kontrolle gibt, die Kodierung von vier expliziten Weiterleitungszweigen, die jeweils laut scheitern statt zu raten, die vollständige Entfernung lokaler Runtime-Fallbacks aus dem Standardpfad und die Verankerung der Regel durch ein Build-Gate sowie einen Mutationstest, der den Build fehlschlagen lässt, wenn die Garantie jemals entfernt wird.
- **„Grüne Tests, kaputte Funktionen.“** Die Verfassung benennt diesen Fehlermodus direkt und bekämpft ihn mit dem Anti-Bluff-Testregime: konkrete, nutzersichtbare Assertions statt Implementierungsdetails, echte Systeme im Einsatz (Mocks nur in Unit-Tests), obligatorische Mutationstests (bewusstes Brechen der Funktion muss den Test *rot* werden lassen) und visuelle QS, die das Ergebnis tatsächlich prüft.
- **Langformatige, multimediale Qualität.** Buchlange Eingaben belasten sowohl die Konsistenz als auch das Budget; gelöst durch mehrstufiges Polieren, das den Text erneut prüft, eine Vorbereitungsphase, die den Arbeitsaufwand vorab einschätzt, und Übersetzungscaching, das verhindert, dass derselbe Abschnitt zweimal bezahlt werden muss.

Technologie-Stack

- **Go** — ausgewählt wegen seiner Nebenläufigkeitsprimitive, die sich natürlich auf das gleichzeitige Parsen, Übersetzen und Streamen vieler Kapitel abbilden lassen; das hochgradig nebenläufige Backend, Modul `digital.vasic.translator`.
- **Gin** — ausgewählt als schneller, minimalistischer HTTP-Router für die Bereitstellung der REST-API-Oberfläche.
- **QUIC / HTTP/3 (quic-go)** — ausgewählt, um der REST-API-Oberfläche einen latenzarmen, modernen Transport zu bieten, der auch bei unzuverlässigen Netzwerken stabil bleibt.
- **gRPC + Protocol Buffers** — ausgewählt für eine stark typisierte, leistungsstarke Dienstschnittstelle, die parallel zu REST für programmatische Aufrufer läuft.
- **Gorilla WebSocket** — ausgewählt, um den Echtzeit-Stream typisierter Übersetzungsereignisse zu übertragen, der das Überwachungs-Dashboard live speist.

- **PostgreSQL, SQLite, Redis** — eine bewusste Dreiteilung: PostgreSQL für dauerhafte relationale Daten, SQLite für eingebetteten/lokalen Zustand (dient auch als Backend für den Speicher verifizierter Modelle, `data/verified_models.db`) und Redis als Hot-Cache.
- **unidoc/unioffice + unipdf** — ausgewählt, um die komplexen Formate zu handhaben: DOCX- und PDF-Parsing sowie -Regenerierung, damit E-Books im Mehrformat-Roundtrip zuverlässig funktionieren.
- **Cobra** — ausgewählt als CLI-Framework, das den unified-translator und seine Begleitwerkzeuge antreibt.
- **golang-jwt (JWT HS256)** — ausgewählt für zustandslose API-Authentifizierung, kombiniert mit Token-Bucket-Ratenbegrenzung pro IP und TLS/QUIC-Transportverschlüsselung, um die Oberfläche abzusichern.
- **LLMsVerifier-Brücke (pkg/bridge)** — das zentrale Element: liefert das stärkste verifizierte Modell samt seiner deterministischen Fallback-Kette und dient als einzige Durchsetzungsinstanz für die Garantie, dass keine stillen Fallbacks auftreten.
- **Testify** — ausgewählt für die Go-Testsuite, einschließlich der dedizierten `provider_routing_test.go` und der Mutations-Gates, die die Ehrlichkeitsregeln einhalten.
- **Docker / Podman (rootless) + Compose** — ausgewählt für containerisierte, verteilte Bereitstellung (`docker-compose.distributed.yml`), mit rootless Podman für eine strengere Sicherheitsposition.

Status & Ehrlichkeitshinweise

- **Status: Beta.** Funktionsfähige Plattform; die Version wird in `VERSION/Makefile/AGENTS.md` inkonsistent angegeben und gilt daher als unbestätigt.
- **Lizenz: Unklar.** Die README behauptet MIT, doch dies wurde nicht anhand einer LICENSE-Datei bestätigt — vor einer Angabe verifizieren.
- Die Dashboard-/Monitor-Endpunkte sind nur lokal verfügbar, nicht öffentlich. Die in der Dokumentation genannten WebSocket-Leistungswerte sind Zielvorgaben, keine verifizierten Daten. `ARCHITECTURE.md` listet noch entfernte Ollama/lokale Engines auf (veraltet).

Prioritätsstufe: Helix-primär (LLM-Infrastruktur-Cluster).
Rangiert innerhalb der Helix-Plattformfamilie, nach HelixTrack.